Il Master theorem

Sapienza Università di Roma

Indice

Ricorrenze in forma standard

Il Master theorem

Applicare il Master theorem

Quando il Master theorem non si applica

Dimostrazione del Master theorem

Introduzione (1/2)

- Nel capitolo *Equazioni di ricorrenza* abbiamo risolto ricorrenze per espansione, per sostituzione e con gli alberi di ricorsione
- Ogni metodo richiede qualche calcolo, e per le ricorrenze divide et impera il calcolo è sempre lo stesso:
  - disegnare l'albero
  - sommare i costi dei livelli
  - confrontare la radice con le foglie
- Alla fine di quel capitolo abbiamo osservato tre possibili comportamenti: dominano le **foglie**, tutti i **livelli** costano uguale, oppure domina la **radice**

Introduzione (2/2)

Il **Master theorem** trasforma questi tre comportamenti in un risultato pronto all'uso. In questo capitolo

- richiamiamo le ricorrenze in forma standard e gli algoritmi che le producono
- enunciamo il teorema e ne spieghiamo i tre casi
- lo applichiamo con uno schema fisso in cinque passi a molti esempi, e mostriamo le situazioni in cui non può essere applicato
- lo dimostriamo, prima per potenze esatte di b e poi per ogni numero naturale

Ricorrenze in forma standard

Introduzione: Ricorrenze in forma standard

- In questa sezione richiamiamo la forma delle ricorrenze prodotte dagli algoritmi divide et impera: la **forma standard**
- Diamo un nome ai suoi tre ingredienti: il numero di sottoproblemi a, il fattore b che riduce la dimensione, e il costo $f(n)$ di dividere e combinare
- Questi tre ingredienti sono tutto ciò che serve al Master theorem per dare l'ordine di grandezza della soluzione

Equazioni di ricorrenza

- Nel capitolo *Equazioni di ricorrenza* abbiamo visto che il numero di passi eseguiti da un algoritmo ricorsivo è descritto naturalmente da una **equazione di ricorrenza**

- Un'equazione di ricorrenza esprime il valore di una funzione su un input di dimensione n in termini dei suoi valori su input più piccoli

- Una grande famiglia di algoritmi *divide et impera* produce ricorrenze di una forma molto specifica, detta **forma standard**:

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n)$$

La forma standard

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n)$$

dove:

- n è la dimensione dell'input;
- a è il **numero di sottoproblemi** generati da ciascuna chiamata ricorsiva ($a \geq 1$, costante);
- $\frac{n}{b}$ è la **dimensione di ciascun sottoproblema** ($b > 1$, costante); quando n/b non è un numero naturale si intende $\lfloor n/b \rfloor$ oppure $\lceil n/b \rceil$;
- $f(n)$ è il **costo di dividere** il problema negli a sottoproblemi **e di combinare** le loro soluzioni (tutto ciò che viene fatto al di fuori delle chiamate ricorsive)

Positività di $f(n)$ e caso base

- Supponiamo sempre che $f(n)$ sia **asintoticamente positiva**, cioè

$$f(n) > 0 \quad \text{per ogni } n \text{ sufficientemente grande}$$

- Per n sufficientemente piccolo (per esempio $n = 1$) la ricorsione si ferma e

$$T(n) = \Theta(1) :$$

risolvere un problema di dimensione costante richiede un numero costante di passi

- Per questo motivo il caso base di solito non viene nemmeno scritto

Una ricorrenza come oggetto matematico

- In questo capitolo una ricorrenza è un **oggetto matematico**: usiamo una generica funzione $T(n)$
- Quando analizziamo un algoritmo ricorsivo, la funzione descritta dalla ricorrenza è il numero di passi $S(n)$ eseguiti nel caso peggiore su un input di parametro di dimensione n
- Il tempo di esecuzione nel caso peggiore $T(|I|)$ si ottiene poi da $S(n)$ esattamente come nel capitolo *Tempo di esecuzione*

Esempi di ricorrenze in forma standard

Saranno tutte risolte più avanti in questo capitolo:

Algoritmo	Ricorrenza	a	b	$f(n)$
Ricerca binaria (ricorsiva)	$T(n) = T(n/2) + 1$	1	2	1
Somma di un array (ricorsiva)	$T(n) = 2T(n/2) + 1$	2	2	1
Merge sort	$T(n) = 2T(n/2) + n$	2	2	n
Moltiplicazione di Karatsuba	$T(n) = 3T(n/2) + n$	3	2	n
Matrici di Strassen	$T(n) = 7T(n/2) + n^2$	7	2	n^2

- Il **Master theorem** fornisce l'ordine di grandezza di $T(n)$ per la maggior parte delle ricorrenze in forma standard, direttamente da a, b e $f(n)$, senza alcun calcolo sulla ricorsione

Riepilogo

Forma standard:

- $T(n) = a\,T(n/b) + f(n)$, con costanti $a \geq 1$ e $b > 1$
- n/b significa $\lfloor n/b \rfloor$ oppure $\lceil n/b \rceil$ quando non è un numero naturale
- $f(n)$ è il costo al di fuori delle chiamate ricorsive, asintoticamente positiva

Caso base:

- $T(n) = \Theta(1)$ per n sufficientemente piccolo, di solito non scritto

Esempi:

- ricerca binaria, somma di un array, merge sort, Karatsuba, Strassen

Il Master theorem

Introduzione: Il Master theorem

- In questa sezione enunciamo il teorema e ne spieghiamo i **tre casi**
- Diamo una regola pratica per ipotizzare il caso, e l'intuizione che c'è dietro: i costi dei livelli dell'albero di ricorsione
- Vediamo anche che i tre casi lasciano dei **vuoti**, in cui il teorema non dà alcuna risposta, e perché la base dei logaritmi conta solo nell'esponente

Il Master theorem (1/2)

Teorema 1: Master theorem

Siano $a \geq 1$ e $b > 1$ costanti, sia $f(n)$ una funzione asintoticamente positiva, e sia $T(n)$ definita sui numeri naturali dalla ricorrenza

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n),$$

dove n/b è interpretato come $\lfloor n/b \rfloor$ oppure $\lceil n/b \rceil$, e $T(n) = \Theta(1)$ per n sufficientemente piccolo. Allora $T(n)$ ha le seguenti limitazioni asintotiche (slide successiva).

Il Master theorem (2/2)

Teorema: Master theorem (continua)

Caso 1: se $f(n) = O\left(n^{\log_b a - \epsilon}\right)$ per qualche costante $\epsilon > 0$, allora $T(n) = \Theta\left(n^{\log_b a}\right)$;

Caso 2: se $f(n) = \Theta\left(n^{\log_b a} \log^k n\right)$ per qualche costante $k \geq 0$, allora $T(n) = \Theta\left(n^{\log_b a} \log^{k+1} n\right)$;

Caso 3: se $f(n) = \Omega\left(n^{\log_b a + \epsilon}\right)$ per qualche costante $\epsilon > 0$, e se

$$a\, f\left(\frac{n}{b}\right) \leq c\, f(n)$$

per qualche costante $c < 1$ e per ogni n sufficientemente grande (***condizione di regolarità***), allora $T(n) = \Theta\left(f(n)\right)$.

Sull'enunciato

- Questo è l'enunciato classico del teorema (come nel libro di testo di Cormen, Leiserson, Rivest e Stein, *Introduction to Algorithms*), con il **caso 2 esteso**
- Per $k = 0$ si ottiene l'enunciato di base:

$$\text{se } f(n) = \Theta(n^{\log_b a}) \text{ allora } T(n) = \Theta(n^{\log_b a} \log n)$$

- Le notazioni O, Ω e Θ dell'enunciato sono quelle definite nel capitolo «Notazione asintotica»
- La dimostrazione è data nell'ultima sezione di questo capitolo

Prima di applicare il teorema, impariamo a ipotizzare rapidamente
in quale dei tre casi ci troviamo.

Regola pratica (1/2)

- In ciascuno dei tre casi confrontiamo la funzione $f(n)$ con la funzione

$$n^{\log_b a},$$

che (come vedremo nella dimostrazione) è il **numero di foglie** dell'albero di ricorsione

- **Intuitivamente, la più grande tra le due funzioni determina la soluzione della ricorrenza**

- Come regola pratica, si controlli la relazione tra $n^{\log_b a}$ e $f(n)$ (slide successiva) e si determini di conseguenza l'ordine di grandezza di $T(n)$ (se possibile)

Regola pratica (2/2)

$$n^{\log_b a} \begin{cases} \text{polinomialmente più grande di } f(n) \\ \qquad \longrightarrow \quad \text{caso 1:} \quad T(n) = \Theta\left(n^{\log_b a}\right) \\ \text{dello stesso ordine di } f(n) \text{ (a meno di } \log^k n) \\ \qquad \longrightarrow \quad \text{caso 2:} \quad T(n) = \Theta\left(n^{\log_b a} \log^{k+1} n\right) \\ \text{polinomialmente più piccolo di } f(n) \\ \qquad \longrightarrow \quad \text{caso 3:} \quad T(n) = \Theta\left(f(n)\right) \end{cases}$$

Che cosa significa "polinomialmente"

- **Polinomialmente** più piccola (più grande) significa più piccola (più grande) **di un fattore** n^ϵ per qualche costante $\epsilon > 0$

- Per esempio:
  - n è polinomialmente più piccolo di n^2 (fattore n^1)
  - n^2 è polinomialmente più piccolo di $n^{2.1}$ (fattore $n^{0.1}$)
  - n è più piccolo di $n \log n$, ma **non** polinomialmente più piccolo: per ogni $\epsilon > 0$

$$\lim_{n\to\infty} \frac{n \log n}{n^{1+\epsilon}} = \lim_{n\to\infty} \frac{\log n}{n^\epsilon} = 0^+,$$

quindi $n \log n$ cresce più lentamente di $n \cdot n^\epsilon$, qualunque $\epsilon > 0$ scegliamo

Intuizione: i costi dei livelli (1/2)

- Srotolando la ricorrenza (con l'albero di ricorsione che disegneremo nella dimostrazione) il costo totale è

$$T(n) = \underbrace{\Theta\left(n^{\log_b a}\right)}_{\text{costo delle foglie}} + \underbrace{\sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)}_{\text{costo dei livelli interni}}$$

  dove il livello j dell'albero contiene a^j sottoproblemi di dimensione n/b^j

- Consideriamo il caso semplice $f(n) = n^d$. Il costo del livello j è

$$a^j \left(\frac{n}{b^j}\right)^d = n^d \left(\frac{a}{b^d}\right)^j,$$

  quindi i costi dei livelli formano una **progressione geometrica di ragione** a/b^d

Intuizione: i costi dei livelli (2/2)

Poiché $a/b^d > 1 \iff d < \log_b a$:

- caso 1 ($d < \log_b a$): i costi dei livelli **crescono** geometricamente dalla radice alle foglie; il totale è dominato dalle foglie, cioè da $n^{\log_b a}$
- caso 2 ($d = \log_b a$): tutti i $\log_b n$ livelli hanno **lo stesso costo** $n^d = n^{\log_b a}$; il totale è $n^{\log_b a}$ per il numero di livelli, cioè $n^{\log_b a} \log n$
- caso 3 ($d > \log_b a$): i costi dei livelli **decrescono** geometricamente; il totale è dominato dalla radice, cioè da $f(n)$

I vuoti tra i casi

- I tre casi **non coprono tutte le possibili funzioni** $f(n)$:
  - un vuoto tra il caso 1 e il caso 2 quando $f(n)$ è più piccola di $n^{\log_b a}$ ma **non polinomialmente più piccola** (e non della forma $n^{\log_b a} \log^k n$ con $k \geq 0$), per esempio $f(n) = n^{\log_b a} / \log n$
  - un vuoto tra il caso 2 e il caso 3 quando $f(n)$ è più grande di $n^{\log_b a}$ ma **non polinomialmente più grande** (e non della forma $n^{\log_b a} \log^k n$), per esempio $f(n) = n^{\log_b a} 2^{\sqrt{\log_2 n}}$
  - il caso 3 non si applica anche quando $f(n)$ è polinomialmente più grande di $n^{\log_b a}$ ma la **condizione di regolarità** non vale

- In queste situazioni il Master theorem **non può essere usato**, e la ricorrenza deve essere risolta con altre tecniche (albero di ricorsione, iterazione, metodo di sostituzione), come vedremo più avanti in questo capitolo

Una nota sulla base dei logaritmi

- Poiché $\log_c n = \dfrac{\log_2 n}{\log_2 c}$ per ogni base $c > 1$, abbiamo

$$\log_c^k n = \frac{1}{\log_2^k c} \log_2^k n = \Theta\big(\log_2^k n\big)$$

- Quindi, **dentro Θ, O e Ω la base del logaritmo è irrilevante**: nel caso 2 scriviamo semplicemente $\log^k n$

- Al contrario, la base b nell'**esponente** $n^{\log_b a}$ è essenziale: per esempio

$$n^{\log_2 4} = n^2 \qquad \text{mentre} \qquad n^{\log_4 4} = n,$$

e $n^2 \neq \Theta(n)$

Il Master theorem in breve

- Il Master theorem confronta il costo della radice, $f(n)$, con il numero di foglie dell'albero di ricorsione, $n^{\log_b a}$: la più grande tra le due determina $T(n)$
- Quando sono dello stesso ordine (a meno di $\log^k n$) compare un ulteriore fattore $\log n$
- "Più grande" e "più piccola" devono essere **polinomiali** (di un fattore n^ϵ); nel caso 3 deve valere anche la **condizione di regolarità**
- Quando $f(n)$ cade in un vuoto tra i casi, il teorema non dà alcuna risposta

Riepilogo

I tre casi, confrontando $f(n)$ con $n^{\log_b a}$:

- caso 1, f polinomialmente più piccola: $T(n) = \Theta(n^{\log_b a})$
- caso 2, $f = \Theta(n^{\log_b a} \log^k n)$: $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$
- caso 3, f polinomialmente più grande e regolare: $T(n) = \Theta(f(n))$

Intuizione:

- i costi dei livelli crescono, restano uguali o decrescono dalla radice alle foglie

Limiti:

- vuoti tra i casi; base dei logaritmi essenziale solo in $n^{\log_b a}$

Applicare il Master theorem

Introduzione: Applicare il Master theorem

- In questa sezione applichiamo il teorema con uno **schema fisso in cinque passi**
- Cominciamo con tre ricorrenze, una per ciascun caso, e poi risolviamo le ricorrenze degli algoritmi elencati nella sezione precedente
- Una verifica numerica conferma che il teorema dà l'**ordine di grandezza**, non la costante

Lo schema in cinque passi

Per risolvere una ricorrenza con il Master theorem seguiamo sempre questo schema:

1. **identificare** a, b e $f(n)$ (e controllare che $a \geq 1$, $b > 1$ siano costanti e $f(n)$ sia asintoticamente positiva);
2. **calcolare** $n^{\log_b a}$;
3. **confrontare** $n^{\log_b a}$ con $f(n)$ e ipotizzare il caso;
4. **verificare** l'ipotesi del caso: trovare $\epsilon > 0$ (caso 1 o 3), trovare $k \geq 0$ (caso 2), e controllare la condizione di regolarità (caso 3);
5. **concludere**.

Esempio: $T(n) = 16\,T(n/4) + n$

1. $a = 16,\ b = 4,\ f(n) = n$
2. $n^{\log_b a} = n^{\log_4 16} = n^2$
3. n^2 è polinomialmente più grande di $f(n) = n \longrightarrow$ caso 1
4. $f(n) = n = O\left(n^{2-\epsilon}\right)$ con $\epsilon = 1$
5. $T(n) = \Theta\left(n^{\log_4 16}\right) = \Theta(n^2)$

Esempio: $T(n) = 8\,T(n/2) + n^3$

1. $a = 8,\ b = 2,\ f(n) = n^3$
2. $n^{\log_b a} = n^{\log_2 8} = n^3$
3. n^3 e $f(n) = n^3$ hanno lo stesso ordine $\longrightarrow$ caso 2
4. $f(n) = n^3 = \Theta\left(n^3 \log^k n\right)$ con $k = 0$
5. $T(n) = \Theta\left(n^3 \log^{k+1} n\right) = \Theta(n^3 \log n)$

Esempio: $T(n) = 8\,T(n/3) + n^2$ (1/2)

1. $a = 8,\ b = 3,\ f(n) = n^2$
2. $n^{\log_b a} = n^{\log_3 8} \approx n^{1.893}$
3. $n^{1.893}$ è polinomialmente più piccolo di $f(n) = n^2 \longrightarrow$ caso 3
4. $f(n) = n^2 = \Omega\left(n^{\log_3 8 + \epsilon}\right)$ per ogni $0 < \epsilon \leq 2 - \log_3 8 \approx 0.107$
   Condizione di regolarità: si veda la slide successiva
5. $T(n) = \Theta(n^2)$

Esempio: $T(n) = 8\,T(n/3) + n^2$ (2/2)

Condizione di regolarità (passo 4): cerchiamo una costante $c < 1$ tale che, per ogni n grande,

$$8\,f\left(\frac{n}{3}\right) \leq c\,f(n)$$

$$\iff \quad 8\left(\frac{n}{3}\right)^2 \leq c\,n^2$$

$$\iff \quad \frac{8}{9}\,n^2 \leq c\,n^2,$$

che vale per ogni c con $\frac{8}{9} \leq c < 1$, per esempio $c = \frac{8}{9}$

Quindi si applica il caso 3 e $T(n) = \Theta(n^2)$

Risolviamo ora le ricorrenze degli algoritmi elencati all'inizio del capitolo.

Esempio: ricerca binaria, $T(n) = T(n/2) + 1$

1. $a = 1,\ b = 2,\ f(n) = 1$
2. $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$
3. 1 e $f(n) = 1$ hanno lo stesso ordine $\longrightarrow$ caso 2
4. $f(n) = 1 = \Theta\left(n^0 \log^k n\right)$ con $k = 0$
5. $T(n) = \Theta\left(n^0 \log n\right) = \Theta(\log n)$

Esempio: merge sort, $T(n) = 2\,T(n/2) + n$

1. $a = 2,\ b = 2,\ f(n) = n$
2. $n^{\log_b a} = n^{\log_2 2} = n$
3. n e $f(n) = n$ hanno lo stesso ordine $\longrightarrow$ caso 2
4. $f(n) = n = \Theta\left(n \log^k n\right)$ con $k = 0$
5. $T(n) = \Theta(n \log n)$

Esempio: somma di un array, $T(n) = 2\,T(n/2) + 1$ (1/2)

1. $a = 2,\ b = 2,\ f(n) = 1$
2. $n^{\log_b a} = n^{\log_2 2} = n$
3. n è polinomialmente più grande di $f(n) = 1 \longrightarrow$ caso 1
4. $f(n) = 1 = O\left(n^{1-\epsilon}\right)$ con $\epsilon = 1$ (infatti $n^{1-1} = n^0 = 1$)
5. $T(n) = \Theta(n)$

Esempio: somma di un array, $T(n) = 2\,T(n/2) + 1$ (2/2)

- La somma ricorsiva di un array che divide l'array in due metà (l'algoritmo `RecursiveSum` del capitolo «Algoritmi ricorsivi») è un algoritmo di questo tipo
- Ogni chiamata esegue una quantità costante di lavoro al di fuori delle chiamate ricorsive, e il numero totale di chiamate è $\Theta(n)$
- Contandole (nei notebook Colab del capitolo), su $n = 8$, 1000 e $2^{20} = 1\,048\,576$ elementi otteniamo 15, 1 999 e 2 097 151 chiamate, cioè esattamente $2n - 1$

Esegui in Colab: Python · C++ · Java

Esempio: Karatsuba, $T(n) = 3\,T(n/2) + n$

1. $a = 3,\ b = 2,\ f(n) = n$
2. $n^{\log_b a} = n^{\log_2 3} \approx n^{1.585}$
3. $n^{1.585}$ è polinomialmente più grande di $f(n) = n \longrightarrow$ caso 1
4. $f(n) = n = O\left(n^{\log_2 3 - \epsilon}\right)$ con $\epsilon = \log_2 3 - 1 \approx 0.585 > 0$
5. $T(n) = \Theta\left(n^{\log_2 3}\right) \approx \Theta\left(n^{1.585}\right)$: l'algoritmo di Karatsuba è asintoticamente più veloce della moltiplicazione scolastica $\Theta(n^2)$

Esempio: Strassen, $T(n) = 7\,T(n/2) + n^2$

1. $a = 7,\ b = 2,\ f(n) = n^2$
2. $n^{\log_b a} = n^{\log_2 7} \approx n^{2.807}$
3. $n^{2.807}$ è polinomialmente più grande di $f(n) = n^2 \longrightarrow$ caso 1
4. $f(n) = n^2 = O\left(n^{\log_2 7 - \epsilon}\right)$ con $\epsilon = \log_2 7 - 2 \approx 0.807 > 0$
5. $T(n) = \Theta\left(n^{\log_2 7}\right) \approx \Theta\left(n^{2.807}\right)$, meglio del $\Theta(n^3)$ della moltiplicazione di matrici standard (la cui ricorrenza divide et impera è $T(n) = 8\,T(n/2) + n^2$)

Altre due ricorrenze con un logaritmo in $f(n)$,
e una con b non intero.

Esempio: $T(n) = 2\,T(n/2) + n \log n$

1. $a = 2,\ b = 2,\ f(n) = n \log n$
2. $n^{\log_b a} = n^{\log_2 2} = n$
3. $f(n) = n \log n$ è più grande di n, ma solo di un fattore $\log n$, non polinomialmente $\longrightarrow$ non è il caso 3; ha la forma $n^{\log_b a} \log^k n \longrightarrow$ caso 2
4. $f(n) = n \log n = \Theta\left(n \log^k n\right)$ con $k = 1$
5. $T(n) = \Theta\left(n \log^2 n\right)$

Esempio: $T(n) = 3\,T(n/4) + n \log n$ (1/2)

1. $a = 3,\ b = 4,\ f(n) = n \log n$ (logaritmo in base 2)
2. $n^{\log_b a} = n^{\log_4 3} \approx n^{0.792}$
3. $n^{0.792}$ è polinomialmente più piccolo di $f(n) = n \log n \longrightarrow$ caso 3
4. Poiché $n \log n \geq n$ per $n \geq 2$, abbiamo $f(n) = \Omega\left(n^{\log_4 3 + \epsilon}\right)$ con $\epsilon = 1 - \log_4 3 \approx 0.208$

   Condizione di regolarità: si veda la slide successiva
5. $T(n) = \Theta(n \log n)$

Esempio: $T(n) = 3\,T(n/4) + n \log n$ (2/2)

Condizione di regolarità (passo 4): per ogni $n \geq 4$,

$$3\,f\left(\frac{n}{4}\right) = 3\,\frac{n}{4}\,\log\frac{n}{4} \leq \frac{3}{4}\,n \log n = c\,f(n) \qquad \text{con } c = \frac{3}{4} < 1,$$

perché $\log(n/4) \leq \log n$

Quindi si applica il caso 3 e $T(n) = \Theta(n \log n)$

Esempio: $T(n) = T(2n/3) + 1$

1. $a = 1$, $b = \frac{3}{2}$ (infatti $\frac{2n}{3} = \frac{n}{3/2}$), $f(n) = 1$. Si noti che b non deve necessariamente essere un intero
2. $n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$
3. 1 e $f(n) = 1$ hanno lo stesso ordine $\longrightarrow$ caso 2
4. $f(n) = 1 = \Theta\left(n^0 \log^k n\right)$ con $k = 0$
5. $T(n) = \Theta(\log n)$

Una verifica numerica (1/2)

- Per ciascuna delle ricorrenze precedenti con b intero abbiamo calcolato $T(n)$ esattamente dalla ricorrenza, con $T(1) = 1$ e $n = b^L$
- L'abbiamo divisa per l'ordine di grandezza $g(n)$ dato dal teorema (logaritmi in base 2)
- Il rapporto resta compreso tra due costanti positive, come garantisce il teorema, e si avvicina al limite indicato nell'ultima colonna della tabella (slide successiva)

Una verifica numerica (2/2)

Ricorrenza	$g(n)$	$L = 5$	$L = 10$	$L = 20$	limite
$16\,T(n/4) + n$	n^2	1.3330	1.3333	1.3333	$4/3$
$8\,T(n/2) + n^3$	$n^3 \log n$	1.2000	1.1000	1.0500	1
$8\,T(n/3) + n^2$	n^2	4.5606	6.5364	8.2414	9
$T(n/2) + 1$	$\log n$	1.2000	1.1000	1.0500	1
$2\,T(n/2) + n$	$n \log n$	1.2000	1.1000	1.0500	1
$2\,T(n/2) + 1$	n	1.9688	1.9990	2.0000	2
$3\,T(n/2) + n$	$n^{\log_2 3}$	2.7366	2.9653	2.9994	3
$7\,T(n/2) + n^2$	$n^{\log_2 7}$	2.2521	2.3284	2.3333	$7/3$
$2\,T(n/2) + n \log n$	$n \log^2 n$	0.6400	0.5600	0.5275	$1/2$
$3\,T(n/4) + n \log n$	$n \log n$	2.1933	2.8704	3.4020	4

Leggere la tabella

- Per il merge sort, per esempio, $T(2^L) = 2^L(L+1)$, quindi

$$T(n) = n \log_2 n + n \qquad \text{e} \qquad \frac{T(n)}{n \log_2 n} = 1 + \frac{1}{L}$$

- Nel caso 3 la convergenza è lenta: il rapporto tende a $\frac{1}{1-c}$ (9 per $c = \frac{8}{9}$, 4 per $c = \frac{3}{4}$), la limitazione che compare nella dimostrazione del teorema

- Il teorema dà l'**ordine di grandezza**, non la costante: nella tabella i rapporti tendono a costanti diverse, ma non crescono né svaniscono mai

Riepilogo

Cinque passi:

- identificare $a, b, f(n)$; calcolare $n^{\log_b a}$; confrontare e ipotizzare il caso; verificare (ϵ, k, regolarità); concludere

Risultati:

- caso 1: $16T(n/4) + n$, somma di un array, Karatsuba, Strassen
- caso 2: $8T(n/2) + n^3$, ricerca binaria, merge sort, $2T(n/2) + n \log n$, $T(2n/3) + 1$
- caso 3: $8T(n/3) + n^2$, $3T(n/4) + n \log n$

Verifica numerica:

- $T(n)/g(n)$ resta compreso tra due costanti positive

Quando il Master theorem non si applica

Introduzione: Quando il Master theorem non si applica

- Prima di applicare il teorema, si controlli sempre che valgano **tutte** le sue ipotesi
- In questa sezione vediamo situazioni tipiche in cui il teorema **non può** essere applicato: un vuoto tra i casi, ricorrenze non in forma standard, a non costante o minore di 1, una condizione di regolarità che non vale
- Quando possibile, risolviamo la ricorrenza con un'altra tecnica

Esempio: un vuoto, $T(n) = 2\,T(n/2) + n/\log_2 n$ (1/2)

- $a = 2$, $b = 2$, $f(n) = n/\log_2 n$, e $n^{\log_b a} = n$
- **Non è il caso 1**: $f(n)$ è più piccola di n, ma non polinomialmente più piccola. Per ogni $\epsilon > 0$

$$\lim_{n\to\infty} \frac{n/\log_2 n}{n^{1-\epsilon}} = \lim_{n\to\infty} \frac{n^{\epsilon}}{\log_2 n} = +\infty,$$

  quindi $f(n) \neq O(n^{1-\epsilon})$
- **Non è il caso 2**: $f(n) = n\log^{-1} n$ richiederebbe $k = -1 < 0$
- **Non è il caso 3**: $f(n)$ è più piccola di n

Esempio: un vuoto, $T(n) = 2\,T(n/2) + n/\log_2 n$ (2/2)

Soluzione con l'albero di ricorsione (come nella dimostrazione del teorema), per $n = 2^L$ e $T(1) = 1$: il livello j costa

$$2^j \frac{n/2^j}{\log_2(n/2^j)} = \frac{n}{L-j},$$

quindi

$$T(n) = n + \sum_{j=0}^{L-1} \frac{n}{L-j} = n + n\sum_{i=1}^{L} \frac{1}{i} = n\,(1 + H_L),$$

dove $H_L = \sum_{i=1}^{L} \frac{1}{i}$ è l'L-esimo numero armonico. Poiché $\ln(L+1) \leq H_L \leq 1 + \ln L$ (confronto con $\int \frac{dx}{x}$), $H_L = \Theta(\log L)$ e

$$T(n) = \Theta(n \log\log n)$$

Esempio: non in forma standard, $T(n) = T(n-1) + n$

- Il sottoproblema ha dimensione $n-1$, non n/b con $b > 1$ costante: la ricorrenza **non è in forma standard**
- Soluzione per iterazione (con $T(1) = 1$):

$$T(n) = n + (n-1) + \cdots + 2 + T(1) = \sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2)$$

Esempio: sottoproblemi di dimensioni diverse

$$T(n) = T(n/3) + T(2n/3) + n$$

- I due sottoproblemi hanno **dimensioni diverse** $n/3$ e $2n/3$: la ricorrenza non è della forma $a\,T(n/b) + f(n)$
- Albero di ricorsione: ogni livello completo ha dimensione totale n, quindi costo n
- Il cammino più corto dalla radice a una foglia (sempre $n/3$) ha lunghezza $\log_3 n$, il più lungo (sempre $2n/3$) ha lunghezza $\log_{3/2} n$
- Pertanto i primi $\log_3 n$ livelli sono completi (costo $\geq n \log_3 n$), e ci sono al più $\log_{3/2} n + 1$ livelli, ciascuno di costo al più n. Quindi

$$T(n) = \Theta(n \log n)$$

Esempio: ipotesi su a

a **non è costante**: $T(n) = 2^n\, T(n/2) + n^n$

- Il numero di sottoproblemi $a = 2^n$ **dipende da** n: il Master theorem richiede che a sia una costante

$a < 1$: $T(n) = 0.5\, T(n/2) + 1/n$

- Qui $a = 0.5 < 1$: il teorema richiede $a \geq 1$ (deve esserci almeno un sottoproblema)

Esempio: la condizione di regolarità non vale (1/3)

$$T(n) = T(n/2) + n(2 - \cos n)$$

- $a = 1$, $b = 2$, $n^{\log_b a} = n^0 = 1$, $f(n) = n(2 - \cos n)$
- Poiché $1 \leq 2 - \cos n \leq 3$, abbiamo $n \leq f(n) \leq 3n$, quindi

$$f(n) = \Theta(n) = \Omega\left(n^{0+\epsilon}\right) \quad \text{con } \epsilon = 1$$

- $f(n)$ è polinomialmente più grande di $n^{\log_b a}$, e ci aspetteremmo il caso 3

Esempio: la condizione di regolarità non vale (2/3)

La condizione di regolarità non vale. Prendiamo $n = 2m$ pari. Allora

$$\frac{a\, f(n/2)}{f(n)} = \frac{m\,(2 - \cos m)}{2m\,(2 - \cos 2m)} = \frac{2 - \cos m}{2\,(2 - \cos 2m)}$$

- Se $\cos m \leq -0.9$, allora $2 - \cos m \geq 2.9$ e $\cos 2m = 2\cos^2 m - 1 \geq 2 \cdot 0.81 - 1 = 0.62$, quindi $2(2 - \cos 2m) \leq 2.76$ e il rapporto è > 1
- Poiché π è irrazionale, i valori m mod 2π ($m \in \mathbb{N}$) sono densi in $[0, 2\pi)$, quindi esistono **infiniti** m con $\cos m \leq -0.9$ (per esempio $m = 3, 9, 16, 22, 28, \ldots$)
- Per $m = 22$: $\cos 22 \approx -0.99996$, $\cos 44 \approx 0.99984$, e il rapporto è ≈ 1.4997

Esempio: la condizione di regolarità non vale (3/3)

- Quindi non esiste alcuna costante $c < 1$ tale che $af(n/2) \leq cf(n)$ per ogni n grande: il **caso 3 non può essere applicato**

- Ciononostante la ricorrenza può essere risolta direttamente (con $n/2$ che significa $\lfloor n/2 \rfloor$): srotolandola,

$$n \leq f(n) \leq T(n) \leq \sum_{j \geq 0} 3 \frac{n}{2^j} + T(1) = 6n + T(1),$$

quindi

$$T(n) = \Theta(n)$$

Riepilogo

Il teorema non si può applicare quando:

- $f(n)$ cade in un vuoto: $2T(n/2) + n/\log_2 n = \Theta(n \log\log n)$
- la ricorrenza non è in forma standard: $T(n-1) + n = \Theta(n^2)$
- i sottoproblemi hanno dimensioni diverse:
  $T(n/3) + T(2n/3) + n = \Theta(n \log n)$
- a non è una costante, oppure $a < 1$
- la condizione di regolarità non vale:
  $T(n/2) + n(2 - \cos n) = \Theta(n)$

Altre tecniche:

- albero di ricorsione, iterazione, metodo di sostituzione

Dimostrazione del Master theorem

Introduzione: Dimostrazione del Master theorem

- In questa sezione dimostriamo il teorema; la dimostrazione spiega **perché** valgono i tre casi

- Seguiamo la dimostrazione classica, che ha due parti:
  1. supponiamo prima che n sia una **potenza esatta di** b, cioè $n = b^L$ per qualche $L \in \mathbb{N}$; allora tutte le dimensioni dei sottoproblemi $n/b, n/b^2, \ldots$ sono potenze esatte di b e non servono parti intere inferiori o superiori (tre lemmi)
  2. estendiamo poi il risultato a ogni numero naturale n, quando n/b è $\lceil n/b \rceil$ oppure $\lfloor n/b \rfloor$

Impostazione (1/2)

- In tutta la dimostrazione poniamo

$$p = \log_b a, \qquad \text{in modo che} \qquad b^p = a \quad \text{e} \quad p \geq 0$$

(poiché $a \geq 1$)

- Per n potenza esatta di b consideriamo la ricorrenza

$$T(n) = \begin{cases} d & \text{se } n = 1, \\ a\,T(n/b) + f(n) & \text{se } n = b^i,\ i \geq 1, \end{cases} \tag{1}$$

dove $d > 0$ è una costante (questo è il significato di $T(1) = \Theta(1)$)

Impostazione (2/2)

- Supponiamo $f(m) > 0$ per ogni $m \geq 1$
  - questo è sempre vero quando f conta i passi di un algoritmo
  - in generale, poiché f è asintoticamente positiva, solo un numero finito di valori è coinvolto e cambiano solo le costanti della dimostrazione

- Le ipotesi dei tre casi valgono "per ogni n sufficientemente grande", cioè per $n \geq n_0$

- La seguente osservazione ci permette di usarle per **ogni** potenza esatta di b almeno uguale a b

Osservazione 1: dalle potenze grandi a tutte le potenze

Siano $f(m) > 0$ e $h(m) > 0$ per ogni potenza esatta $m \geq b$ di b. Se esistono costanti $c > 0$ e n_0 tali che $f(m) \leq c\, h(m)$ (risp. $f(m) \geq c\, h(m)$) per tutte le potenze esatte $m \geq n_0$, allora esiste una costante $c' > 0$ tale che $f(m) \leq c'\, h(m)$ (risp. $f(m) \geq c'\, h(m)$) per ***tutte*** *le potenze esatte $m \geq b$ di b.*

Dimostrazione

Ci sono solo un numero finito di potenze esatte di b nell'intervallo $[b, n_0)$. Sia

$$c' = \max\left\{c, \max_{b \le m < n_0} \frac{f(m)}{h(m)}\right\}$$

$$\left(\text{risp. } c' = \min\left\{c, \min_{b \le m < n_0} \frac{f(m)}{h(m)}\right\} > 0\right),$$

dove m varia tra le potenze esatte di b. Allora $f(m) \le c' h(m)$ (risp. $\ge$):

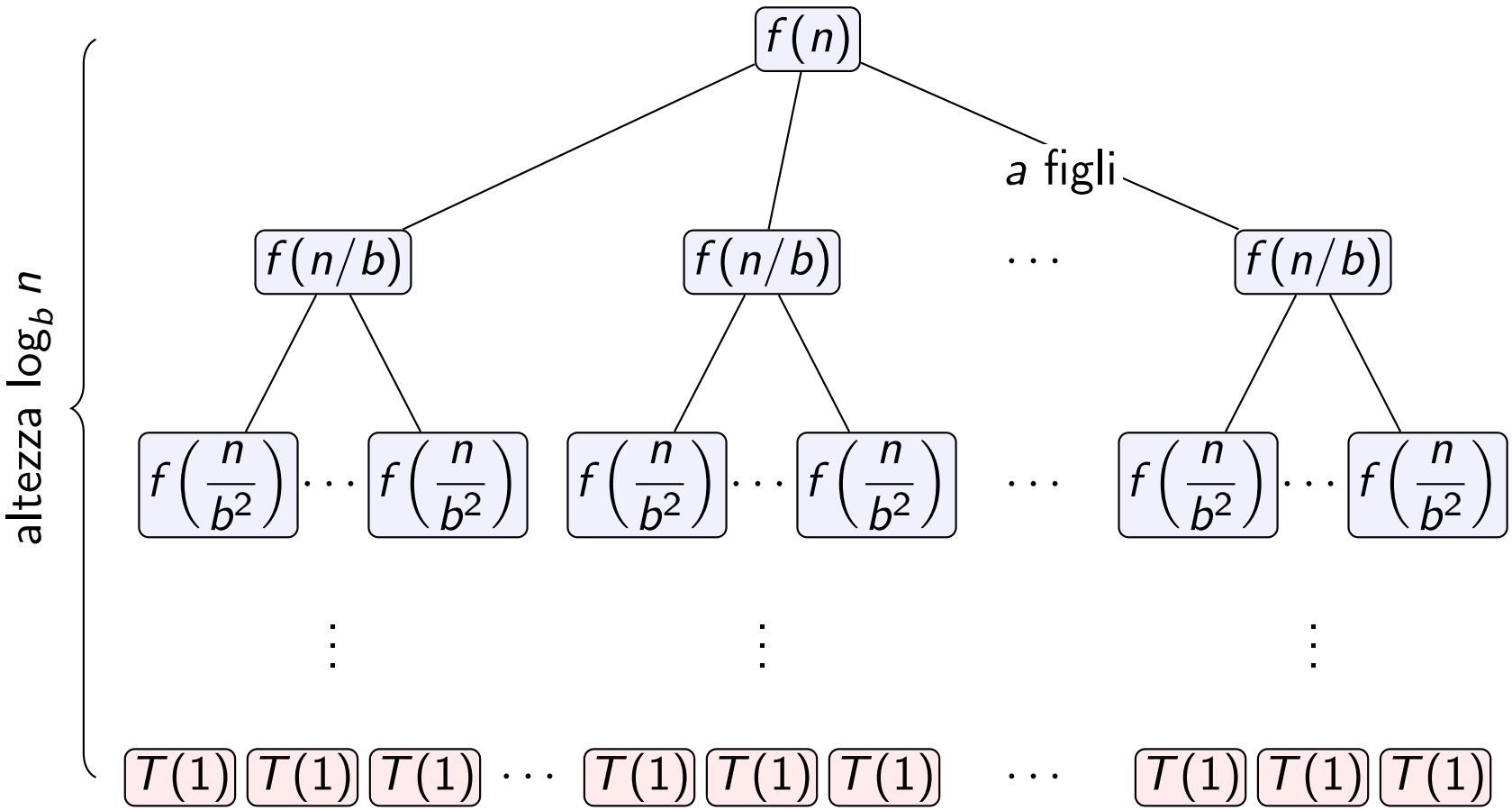
- per $m < n_0$, per la scelta di c'
- per $m \ge n_0$, poiché $c \le c'$ (risp. $c \ge c'$)

□

- La applicheremo con $h(m) = m^{p-\epsilon}$ e $h(m) = m^p \log_b^k m$, che sono positive per $m \ge b$ (si noti che $\log_b m \ge 1$ per $m \ge b$)

Disegniamo ora l'albero di ricorsione della ricorrenza
e calcoliamone il costo totale.

L'albero di ricorsione (1/2)



L'albero di ricorsione (2/2)

Albero di ricorsione di $T(n) = aT(n/b) + f(n)$ per n potenza esatta di b: ogni nodo interno ha a figli; il livello j ha a^j nodi, ciascuno un sottoproblema di dimensione n/b^j e costo $f(n/b^j)$; le foglie sono a profondità $\log_b n$.

profondità	**numero di nodi**	**costo del livello**
0	1	$f(n)$
1	a	$a\,f(n/b)$
2	a^2	$a^2 f(n/b^2)$
$\vdots$	$\vdots$	$\vdots$
$\log_b n$	$a^{\log_b n}$	$\Theta\left(n^{\log_b a}\right)$

totale: $\Theta\left(n^{\log_b a}\right) + \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j)$

Leggere l'albero

- La radice (livello 0) è un problema di dimensione n; costa $f(n)$ e genera a sottoproblemi di dimensione n/b
- Per induzione, il livello j contiene a^j nodi, ciascuno un sottoproblema di dimensione n/b^j e costo $f(n/b^j)$: il costo totale del livello j è
$$a^j f(n/b^j)$$
- La dimensione del sottoproblema raggiunge 1 quando $n/b^j = 1$, cioè a profondità $j = \log_b n$: le foglie sono gli $a^{\log_b n}$ nodi del livello $\log_b n$, ciascuno di costo $T(1) = d$

Lemma 1: Costo dell'albero di ricorsione

Siano $a \geq 1$ e $b > 1$ costanti e sia $f(n) > 0$ per $n \geq 1$. Per ogni n potenza esatta di b, la soluzione della ricorrenza (1) è

$$T(n) = d\, n^{\log_b a} + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

$$= \Theta\left(n^{\log_b a}\right) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right).$$

Dimostrazione (1/3)

Sia $n = b^L$. Dimostriamo prima per induzione su $i \in \{0, 1, \ldots, L\}$ che

$$T(n) = a^i \, T\left(\frac{n}{b^i}\right) + \sum_{j=0}^{i-1} a^j f\left(\frac{n}{b^j}\right). \tag{2}$$

- Per $i = 0$ la somma è vuota e (2) diventa $T(n) = T(n)$
- Supponiamo che (2) valga per qualche $i < L$. Poiché $n/b^i = b^{L-i}$ con $L - i \geq 1$, la ricorrenza dà

$$T(n/b^i) = a \, T(n/b^{i+1}) + f(n/b^i)$$

Dimostrazione (2/3)

Sostituendo in (2):

$$T(n) = a^i \left(a\, T\left(\tfrac{n}{b^{i+1}}\right) + f\left(\tfrac{n}{b^i}\right) \right) + \sum_{j=0}^{i-1} a^j f\left(\tfrac{n}{b^j}\right)$$

$$= a^{i+1}\, T\left(\tfrac{n}{b^{i+1}}\right) + \sum_{j=0}^{i} a^j f\left(\tfrac{n}{b^j}\right),$$

che è (2) per $i+1$.

Per $i = L = \log_b n$ si ha $T(n/b^L) = T(1) = d$, quindi

$$T(n) = d\, a^{\log_b n} + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

Dimostrazione (3/3)

Resta da mostrare che $a^{\log_b n} = n^{\log_b a}$. Scrivendo $a = b^{\log_b a}$ e $n = b^{\log_b n}$,

$$\begin{aligned} a^{\log_b n} &= \left(b^{\log_b a}\right)^{\log_b n} \\ &= b^{\log_b a \cdot \log_b n} \\ &= \left(b^{\log_b n}\right)^{\log_b a} \\ &= n^{\log_b a} \end{aligned}$$

Infine, $d > 0$ è una costante, quindi $d\, n^{\log_b a} = \Theta(n^{\log_b a})$. □

Foglie e nodi interni

- Il primo termine è il costo delle **foglie**: ci sono esattamente $n^{\log_b a}$ foglie
- La somma è il costo dei **nodi interni** (livelli $0, \ldots, \log_b n - 1$)

Verifichiamo il lemma su un'istanza piccola.

Esempio: il lemma per $T(n) = 2T(n/2) + n$ (1/2)

Prendiamo $T(1) = 1$ e $n = 8$. Qui $a = 2$, $b = 2$, $f(n) = n$, $d = 1$, $\log_2 8 = 3$.

livello j	nodi a^j	dimensione n/b^j	costo per nodo	costo del livello
0	1	8	$f(8) = 8$	8
1	2	4	$f(4) = 4$	8
2	4	2	$f(2) = 2$	8
3 (foglie)	$8 = 8^{\log_2 2}$	1	$T(1) = 1$	8
			totale	32

Esempio: il lemma per $T(n) = 2T(n/2) + n$ (2/2)

- Il lemma dà

$$T(8) = 1 \cdot 8^{\log_2 2} + (8 + 8 + 8) = 8 + 24 = 32$$

- Direttamente dalla ricorrenza:

$$T(2) = 2 \cdot 1 + 2 = 4,$$
$$T(4) = 2 \cdot 4 + 4 = 12,$$
$$T(8) = 2 \cdot 12 + 8 = 32. \quad \checkmark$$

Limitare la somma

- Per il lemma sul costo dell'albero di ricorsione, per risolvere la ricorrenza ci serve l'ordine di grandezza di

$$g(n) = \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

- Un'**identità chiave**, usata molte volte: poiché $b^p = a$,

$$a^j \left(\frac{n}{b^j}\right)^p = \frac{a^j}{(b^p)^j} n^p = \frac{a^j}{a^j} n^p = n^p \qquad \text{per ogni } j \qquad (3)$$

- Nella somma, $j \leq \log_b n - 1$, quindi le dimensioni $m = n/b^j$ sono potenze esatte di b con $m \geq b$: per l'osservazione «dalle potenze grandi a tutte le potenze», le limitazioni su f dei casi 1 e 2 si possono usare per tutte

Lemma 2: Limitazioni sulla somma

Siano $a \geq 1$ *e* $b > 1$ *costanti,* $p = \log_b a$*, sia* $f(m) > 0$ *per* $m \geq 1$*, e sia* $g(n)$ *definita, per* n *potenza esatta di* b*, come sopra. Allora:*

1. *se* $f(n) = O(n^{p-\epsilon})$ *per qualche costante* $\epsilon > 0$*, allora* $g(n) = O(n^p)$*;*
2. *se* $f(n) = \Theta(n^p \log^k n)$ *per qualche costante* $k \geq 0$*, allora* $g(n) = \Theta(n^p \log^{k+1} n)$*;*
3. *se* $f(n) = \Omega(n^{p+\epsilon})$ *per qualche costante* $\epsilon > 0$*, e a* $f(n/b) \leq c\, f(n)$ *per qualche costante* $c < 1$ *e per tutte le potenze esatte* $n \geq n_0$*, allora* $g(n) = \Theta(f(n))$*.*

Dimostrazione del caso 1 (1/2)

Per l'osservazione «dalle potenze grandi a tutte le potenze» esiste una costante $c > 0$ tale che $f(m) \leq c\, m^{p-\epsilon}$ per ogni potenza esatta $m \geq b$. Quindi, usando (3),

$$g(n) \leq c \sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{p-\epsilon}$$

$$= c \sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{p} \left(\frac{b^j}{n}\right)^{\epsilon}$$

$$= c\, n^{p-\epsilon} \sum_{j=0}^{\log_b n - 1} \left(b^{\epsilon}\right)^j$$

Dimostrazione del caso 1 (2/2)

Questa è una serie geometrica finita di ragione $b^\epsilon > 1$ (poiché $b > 1$ e $\epsilon > 0$):

$$\sum_{j=0}^{\log_b n - 1} \left(b^\epsilon\right)^j = \frac{\left(b^\epsilon\right)^{\log_b n} - 1}{b^\epsilon - 1} = \frac{\left(b^{\log_b n}\right)^\epsilon - 1}{b^\epsilon - 1} = \frac{n^\epsilon - 1}{b^\epsilon - 1}$$

Pertanto

$$0 < g(n) \leq c\, n^{p-\epsilon}\, \frac{n^\epsilon - 1}{b^\epsilon - 1} \leq \frac{c}{b^\epsilon - 1}\, n^p,$$

e poiché $\frac{c}{b^\epsilon - 1}$ è una costante positiva, $g(n) = O(n^p)$. □

Esempio: caso 1 del lemma

- Ricorrenza $T(n) = 16T(n/4) + n$, con $n = 64$. Qui $p = \log_4 16 = 2$, $f(m) = m = m^{2-1}$, quindi $c = 1$, $\epsilon = 1$
- I livelli interni costano

$$g(64) = 1 \cdot 64 + 16 \cdot 16 + 256 \cdot 4 = 64 + 256 + 1024 = 1344$$

- I costi crescono del fattore $b^\epsilon = 4$ da un livello al successivo (sono dominati dai livelli più profondi)
- La formula della dimostrazione dà esattamente $n^{p-\epsilon}\frac{n^\epsilon - 1}{b^\epsilon - 1} = 64 \cdot \frac{63}{3} = 1344$, e la limitazione $\frac{c}{b^\epsilon - 1} n^p = \frac{4096}{3} \approx 1365.3$ vale
- Aggiungendo le foglie, $T(64) = 4096 + 1344 = 5440$ con $T(1) = 1$

Dimostrazione del caso 2 (1/3)

Poiché $\log^k n$ e $\log_b^k n$ differiscono per un fattore costante, l'ipotesi è $f(n) = \Theta(n^p \log_b^k n)$. Per l'osservazione «dalle potenze grandi a tutte le potenze» esistono costanti $c_1, c_2 > 0$ tali che

$$c_1 m^p \log_b^k m \leq f(m) \leq c_2 m^p \log_b^k m$$

per ogni potenza esatta $m \geq b$. Sia $L = \log_b n$. Per il termine j della somma, $m = n/b^j$ e $\log_b m = L - j$; per (3),

$$a^j \left(\frac{n}{b^j}\right)^p \log_b^k \frac{n}{b^j} = n^p (L - j)^k$$

Dimostrazione del caso 2 (2/3)

Quindi, ponendo $i = L - j$ (al variare di j da 0 a $L - 1$, i va da L a 1),

$$c_1\, n^p\, S_k(L) \;\leq\; g(n) \;\leq\; c_2\, n^p\, S_k(L), \qquad \text{dove} \quad S_k(L) = \sum_{i=1}^{L} i^k$$

Dimostriamo ora che $\left(\frac{L}{2}\right)^{k+1} \leq S_k(L) \leq L^{k+1}$, cioè $S_k(L) = \Theta(L^{k+1})$:

- *limitazione superiore*: ciascuno dei L termini soddisfa $i^k \leq L^k$ (poiché $k \geq 0$), quindi $S_k(L) \leq L \cdot L^k = L^{k+1}$
- *limitazione inferiore*: teniamo solo i termini con $i \geq \lceil L/2 \rceil$ (tutti i termini sono non negativi). Ce ne sono $L - \lceil L/2 \rceil + 1 \geq L/2$ e ciascuno è almeno $(L/2)^k$. Quindi $S_k(L) \geq \frac{L}{2}\left(\frac{L}{2}\right)^k = \left(\frac{L}{2}\right)^{k+1}$

Dimostrazione del caso 2 (3/3)

Pertanto

$$\frac{c_1}{2^{k+1}}\, n^p \log_b^{k+1} n \;\leq\; g(n) \;\leq\; c_2\, n^p \log_b^{k+1} n,$$

cioè

$$g(n) = \Theta(n^p \log_b^{k+1} n) = \Theta(n^p \log^{k+1} n)$$

□

Esempio: caso 2 del lemma, $n = 16$

Ricorrenze $2T(n/2) + n$ e $2T(n/2) + n \log_2 n$. Qui $p = \log_2 2 = 1$ e $L = \log_2 16 = 4$.

- $f(n) = n$ $(k = 0)$: ogni livello costa $2^j \cdot \frac{16}{2^j} = 16 = n^p$, quindi

$$g(16) = 4 \cdot 16 = 64 = n\, S_0(4), \qquad \text{con } S_0(4) = 4 = L$$

- $f(n) = n \log_2 n$ $(k = 1)$: il livello j costa $n^p(L - j) = 16\,(4 - j)$:

$$g(16) = 64 + 48 + 32 + 16 = 160 = 16\,(4 + 3 + 2 + 1) = n\, S_1(4)$$

Le limitazioni della dimostrazione danno $\left(\frac{L}{2}\right)^2 = 4 \leq S_1(4) = 10 \leq L^2 = 16$, quindi $64 \leq g(16) \leq 256$

Dimostrazione del caso 3 (1/5)

Limitazione inferiore. Tutti i termini di $g(n)$ sono positivi e il termine $j = 0$ è $f(n)$: quindi $g(n) \geq f(n)$.

Limitazione superiore. Sia $L = \log_b n$. Dimostriamo prima, per induzione su j, che

$$a^j f\left(\frac{n}{b^j}\right) \leq c^j f(n) \qquad \text{per ogni } j \geq 0 \text{ tale che } \frac{n}{b^{j-1}} \geq n_0 \qquad (4)$$

- Per $j = 0$ è un'uguaglianza
- Sia $j \geq 1$ con $n/b^{j-1} \geq n_0$. Allora anche $n/b^{j-2} \geq n_0$, quindi l'ipotesi induttiva vale per $j - 1$

Dimostrazione del caso 3 (2/5)

Applicando la condizione di regolarità alla potenza esatta $n/b^{j-1} \geq n_0$,

$$\begin{aligned} a^j f\left(\frac{n}{b^j}\right) &= a^{j-1}\left[a\, f\left(\frac{n/b^{j-1}}{b}\right)\right] \\ &\leq a^{j-1}\, c\, f\left(\frac{n}{b^{j-1}}\right) \\ &\leq c \cdot c^{j-1} f(n) = c^j f(n) \end{aligned}$$

Dividiamo $g(n)$ nei termini che soddisfano $n/b^{j-1} \geq n_0$ (livelli *alti*) e negli altri (livelli *bassi*).

Dimostrazione del caso 3 (3/5)

Livelli alti: per (4) e la serie geometrica infinita di ragione $0 < c < 1$,

$$\sum_{\text{alti}} a^j f\left(\frac{n}{b^j}\right) \leq f(n) \sum_{j=0}^{\infty} c^j = \frac{f(n)}{1-c}$$

Dimostrazione del caso 3 (4/5)

- **Livelli bassi**: se $n/b^{j-1} < n_0$, allora $m = n/b^j < n_0$ è una potenza esatta di b con $m \geq b$, quindi $f(m) \leq M$, dove M è il massimo di f sulle (finite) potenze esatte di b in $[b, n_0)$
  - inoltre $a^j \leq a^L = n^p$ (lemma sul costo dell'albero di ricorsione)
  - il numero di livelli bassi è al più $h = \lceil \log_b n_0 \rceil$ (corrispondono a $m = b, b^2, \ldots$ con $m < n_0$)

  Quindi

$$\sum_{\text{bassi}} a^j f\left(\frac{n}{b^j}\right) \leq h\, M\, n^p$$

Dimostrazione del caso 3 (5/5)

Infine, $f(n) = \Omega(n^{p+\epsilon})$ implica $n^p = O(f(n))$ (infatti $n^p \leq n^{p+\epsilon}$). Pertanto

$$g(n) \leq \frac{f(n)}{1-c} + hMn^p = O(f(n)),$$

e insieme alla limitazione inferiore

$$g(n) = \Theta(f(n))$$

□

Esempio: caso 3 del lemma

- Ricorrenza $T(n) = 8T(n/3) + n^2$, con $n = 27$. Qui $p = \log_3 8 \approx 1.893$, $c = \frac{8}{9}$ e la condizione di regolarità vale per ogni n (quindi $n_0 = 1$, e non ci sono livelli bassi)

- Il livello j costa $8^j \left(\frac{27}{3^j}\right)^2 = 729 \left(\frac{8}{9}\right)^j$:

$$g(27) = 729 + 648 + 576 = 1953,$$

$$f(27) = 729 \leq g(27) \leq \frac{f(27)}{1 - 8/9} = 6561$$

- I costi dei livelli decrescono geometricamente (sono dominati dalla radice)

- Con $T(1) = 1$: $T(27) = 8^3 + 1953 = 512 + 1953 = 2465$

Con i due lemmi possiamo ora dimostrare il teorema quando n è una potenza esatta di b.

Lemma 3: Master theorem per potenze esatte

I tre casi del Master theorem valgono per la ricorrenza (1), *cioè quando n varia tra le potenze esatte di b.*

Dimostrazione

Per il lemma sul costo dell'albero di ricorsione, $T(n) = \Theta(n^p) + g(n)$, con $g(n) > 0$. Usiamo il lemma sulle limitazioni sulla somma.

- Caso 1: $g(n) = O(n^p)$, quindi $\Theta(n^p) \leq T(n) \leq \Theta(n^p) + O(n^p)$, cioè $T(n) = \Theta(n^p) = \Theta(n^{\log_b a})$
- Caso 2: $g(n) = \Theta(n^p \log^{k+1} n)$. Poiché $n^p = O(n^p \log^{k+1} n)$, le foglie sono trascurabili: $T(n) = \Theta(n^p \log^{k+1} n) = \Theta(n^{\log_b a} \log^{k+1} n)$
- Caso 3: $g(n) = \Theta(f(n))$. Poiché $f(n) = \Omega(n^{p+\epsilon})$, abbiamo $n^p = O(f(n))$, quindi di nuovo le foglie sono trascurabili: $T(n) = \Theta(f(n))$



Resta da togliere l'ipotesi che n sia una potenza esatta di b: parti intere inferiori e superiori.

Parti intere inferiori e superiori (1/3)

- Consideriamo ora ogni numero naturale n. Trattiamo in dettaglio la ricorrenza con le parti intere superiori,

$$\begin{aligned} T(n) &= a\,T\left(\lceil n/b \rceil\right) + f(n) && \text{per } n \geq n', \\ d_1 \leq T(n) &\leq d_2 && \text{per } 1 \leq n < n', \end{aligned} \tag{5}$$

  dove n' e $0 < d_1 \leq d_2$ sono costanti, e $f(n) > 0$ per ogni $n \geq 1$

- Nel caso 3 la condizione di regolarità si legge $a\,f(\lceil n/b \rceil) \leq c\,f(n)$ per ogni $n \geq n_0$

Parti intere inferiori e superiori (2/3)

- **Aumentare la soglia.** I valori $T(m)$ per $1 \leq m < N$ sono un numero finito di numeri positivi per ogni costante $N \geq n'$. Quindi, sostituendo d_1 con il loro minimo e d_2 con il loro massimo, possiamo supporre che la ricorrenza sia usata solo per $n \geq N$, dove scegliamo

$$N \geq \max\left\{n',\ n_0,\ \frac{2b-1}{b-1}\right\}$$

Allora le ipotesi su f (e la condizione di regolarità) valgono per ogni argomento $\geq N$

Parti intere inferiori e superiori (3/3)

- **Dimensioni dei sottoproblemi.** A profondità j dell'albero di ricorsione tutti i sottoproblemi hanno la stessa dimensione n_j, dove

$$n_0^* = n, \qquad n_j = \lceil n_{j-1}/b \rceil \quad (j \geq 1)$$

(scriviamo n_0^* per la dimensione a profondità 0 per evitare confusione con la soglia n_0)

Lemma 4: Dimensioni con le parti intere superiori

Per ogni $j \geq 0$:

$$\frac{n}{b^j} \leq n_j < \frac{n}{b^j} + \frac{b}{b-1}.$$

Dimostrazione

Per induzione su j. Per $j = 0$, $n_0^* = n$. Per $j \geq 1$, usando $x \leq \lceil x \rceil < x + 1$ e l'ipotesi induttiva,

$$n_j \geq \frac{n_{j-1}}{b} \geq \frac{n}{b^j},$$

$$\begin{aligned} n_j &< \frac{n_{j-1}}{b} + 1 \\ &< \frac{1}{b}\left(\frac{n}{b^{j-1}} + \frac{b}{b-1}\right) + 1 \\ &= \frac{n}{b^j} + \frac{1}{b-1} + 1 = \frac{n}{b^j} + \frac{b}{b-1} \end{aligned}$$

□

Lemma 5: Master theorem con le parti intere superiori

I tre casi del Master theorem valgono per la ricorrenza (5)*, per ogni numero naturale n.*

- La dimostrazione ha quattro passi: la profondità dell'albero, lo srotolamento, le limitazioni sulla somma dei livelli interni, la conclusione

Dimostrazione (1/7)

Sia $n \geq N$ e $L = \log_b n$ (un numero reale, non necessariamente intero).

Passo 1: profondità dell'albero. Sia D il più piccolo j tale che $n_j < N$ (la profondità alla quale la ricorsione si ferma).

- Per $j \geq L$, il lemma sulle dimensioni con le parti intere superiori dà $n_j < 1 + \frac{b}{b-1} = \frac{2b-1}{b-1} \leq N$; quindi $D \leq \lceil L \rceil < L + 1$
- Per $j \leq L - \log_b N$, $n_j \geq n/b^j \geq N$; quindi $D > L - \log_b N$

Dunque

$$L - \log_b N < D < L + 1$$

Dimostrazione (2/7)

Passo 2: srotolamento. Esattamente come nella dimostrazione del lemma sul costo dell'albero di ricorsione (la ricorrenza vale a ogni dimensione $n_j \geq N$, cioè per $j < D$),

$$T(n) = a^D \, T(n_D) + \sum_{j=0}^{D-1} a^j f(n_j)$$

Poiché $d_1 \leq T(n_D) \leq d_2$ e, per il Passo 1,

$$\frac{n^p}{N^p} = a^{L-\log_b N} \leq a^D \leq a^{L+1} = a\, n^p,$$

il primo termine è $\Theta(n^p)$.

Dimostrazione (3/7)

Passo 3: limitazione di $g(n) = \sum_{j=0}^{D-1} a^j f(n_j)$. Sia $m_j = n/b^j$. Per $0 \leq j \leq D-1$ si ha $j < L$, quindi $m_j > 1$, e per il lemma sulle dimensioni con le parti intere superiori

$$m_j \leq n_j < m_j + \frac{b}{b-1} < C\, m_j, \qquad C = \frac{2b-1}{b-1}, \tag{6}$$

mentre $n_j \geq N$, quindi le limitazioni su f valgono in n_j. Ricordiamo che $a^j m_j^p = n^p$ per (3).

Dimostrazione (4/7)

- Caso 1: $f(n_j) \leq c\, n_j^{p-\epsilon} \leq c\, C'\, m_j^{p-\epsilon}$ con $C' = \max\{1, C^{p-\epsilon}\}$ (se $p - \epsilon \geq 0$ si usa $n_j < C m_j$, altrimenti $n_j \geq m_j$). Come nel lemma sulle limitazioni sulla somma,

$$
\begin{aligned}
g(n) &\leq c C' n^{p-\epsilon} \sum_{j=0}^{D-1} (b^{\epsilon})^j \\
&\leq c C' n^{p-\epsilon} \frac{b^{\epsilon D}}{b^{\epsilon} - 1} \\
&\leq c C' n^{p-\epsilon} \frac{b^{\epsilon} n^{\epsilon}}{b^{\epsilon} - 1} = O(n^p)
\end{aligned}
$$

Dimostrazione (5/7)

- Caso 2: **Limitazione superiore**:
  $\log_b n_j < \log_b C + \log_b m_j \leq \log_b C + L$, quindi

  $$a^j f(n_j) \leq c_2\, a^j\, C^p m_j^p\, (L + \log_b C)^k = c_2\, C^p\, n^p (L + \log_b C)^k,$$

  e sommando i $D < L + 1$ termini,

  $$g(n) \leq c_2 C^p n^p (L+1)(L + \log_b C)^k = O(n^p \log^{k+1} n)$$

Dimostrazione (6/7)

- Caso 2: **Limitazione inferiore**: $n_j \geq m_j > 1$ dà

$$a^j f(n_j) \geq c_1 a^j m_j^p \log_b^k m_j = c_1 n^p (L-j)^k$$

Per n abbastanza grande $D - 1 \geq L/2$ (perché $D > L - \log_b N$), quindi tenendo solo i termini $0 \leq j \leq L/2$ (almeno $L/2$ termini, ciascuno $\geq (L/2)^k$),

$$g(n) \geq c_1 n^p \left(\frac{L}{2}\right)^{k+1} = \Omega(n^p \log^{k+1} n)$$

Dimostrazione (7/7)

- Caso 3: $g(n) \geq f(n)$ (termine $j = 0$). Per $1 \leq j \leq D-1$ si ha $n_{j-1} \geq N \geq n_0$, quindi la condizione di regolarità dà

$$a\, f(n_j) = a\, f(\lceil n_{j-1}/b \rceil) \leq c\, f(n_{j-1});$$

per induzione (come in (4)) $a^j f(n_j) \leq c^j f(n)$, quindi $g(n) \leq \frac{f(n)}{1-c}$ e $g(n) = \Theta(f(n))$

Passo 4: conclusione. $T(n) = \Theta(n^p) + g(n)$ con le stesse limitazioni su $g(n)$ del lemma sulle limitazioni sulla somma; i tre casi seguono esattamente come nella dimostrazione del Master theorem per potenze esatte. □

Parti intere inferiori (1/2)

- Per $T(n) = a\,T(\lfloor n/b \rfloor) + f(n)$ la dimostrazione è la stessa, con $n_j = \lfloor n_{j-1}/b \rfloor$ e le limitazioni

$$\frac{n}{b^j} - \frac{b}{b-1} < n_j \leq \frac{n}{b^j}$$

(dimostrate come il lemma sulle dimensioni con le parti intere superiori, usando $x - 1 < \lfloor x \rfloor \leq x$)

Parti intere inferiori (2/2)

- Scegliendo anche $N \geq \max\{b, \frac{2b}{b-1}\}$, per $j < D$ si ottiene $m_j \geq n_j \geq N$, quindi $\frac{b}{b-1} \leq \frac{m_j}{2}$ e $\frac{m_j}{2} < n_j \leq m_j$, che sostituisce (6)
- Inoltre $L - \log_b\left(N + \frac{b}{b-1}\right) \leq D \leq L - \log_b N + 1$. Cambiano solo le costanti
- Questo completa la dimostrazione del Master theorem. ■

Esempio: merge sort per ogni n (1/3)

$$T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + n, \qquad T(1) = 1$$

- Questa è l'esatta ricorrenza del merge sort su un array di lunghezza qualunque n
- Non ha la forma $aT(\lceil n/b \rceil) + f(n)$, ma è **compresa** tra due ricorrenze di quella forma:

$$U(n) = 2\,U(\lceil n/2 \rceil) + n, \qquad W(n) = 2\,W(\lfloor n/2 \rfloor) + n,$$

  con $U(1) = W(1) = 1$
- U e W sono non decrescenti (per induzione forte: se $n < n''$ allora $\lceil n/2 \rceil \leq \lceil n''/2 \rceil$ e il termine aggiunto n è minore)

Esempio: merge sort per ogni n (2/3)

- Per induzione forte, $W(n) \leq T(n) \leq U(n)$: per $n \geq 2$, usando che U è non decrescente e $\lfloor n/2 \rfloor \leq \lceil n/2 \rceil$,

  $$T(n) \leq U(\lceil n/2 \rceil) + U(\lfloor n/2 \rfloor) + n \leq 2\, U(\lceil n/2 \rceil) + n = U(n),$$

  e simmetricamente $T(n) \geq 2\, W(\lfloor n/2 \rfloor) + n = W(n)$

Esempio: merge sort per ogni n (3/3)

- Per il Master theorem con le parti intere superiori (caso 2, $k = 0$) e la sua versione con le parti intere inferiori, $U(n) = \Theta(n \log n)$ e $W(n) = \Theta(n \log n)$. Quindi $T(n) = \Theta(n \log n)$ per **ogni** n, non solo per le potenze di 2
- Numericamente: $T(1000) = 10976$ e $T(10^6) = 20951424$, con $\frac{T(n)}{n \log_2 n} \approx 1.101$ e ≈ 1.051, rispettivamente

La condizione di regolarità

- Nel caso 3 l'ipotesi $f(n) = \Omega(n^{p+\epsilon})$ è in realtà implicata dalla condizione di regolarità (per f positiva)
- Questo spiega perché, in pratica, basta controllare la condizione di regolarità

Osservazione 2: la regolarità implica la crescita polinomiale

Siano $a \geq 1$, $b > 1$, $p = \log_b a$, *e sia* $f(m) > 0$ *per ogni potenza esatta* m *di* b. *Se* $a\, f(n/b) \leq c\, f(n)$ *per qualche costante* $0 < c < 1$ *e per tutte le potenze esatte* $n \geq n_0$ *di* b, *allora*

$$f(n) = \Omega\left(n^{p+\epsilon}\right)$$

con

$$\epsilon = \log_b \frac{a}{c} - \log_b a = -\log_b c > 0.$$

Dimostrazione (1/3)

Poiché $0 < c < 1$, $\epsilon = -\log_b c > 0$ e $\frac{a}{c} > a \geq 1$. La condizione di regolarità si può riscrivere come

$$f(n) \geq \frac{a}{c} f\left(\frac{n}{b}\right) \qquad \text{per ogni potenza esatta } n \geq n_0$$

Dimostrazione (2/3)

- Sia $n \geq n_0$ una potenza esatta di b, e sia $j \geq 0$ il numero di volte in cui possiamo applicare questa disuguaglianza, cioè il più piccolo j tale che $m = n/b^j$ soddisfa $m < b\, n_0$
- Quindi $n_0 \leq m < b\, n_0$ quando $n \geq n_0$; ogni dimensione intermedia n/b^i, $i < j$, è $\geq bn_0 \geq n_0$

Applicando la disuguaglianza j volte,

$$f(n) \geq \left(\frac{a}{c}\right)^j f\left(\frac{n}{b^j}\right) \geq \left(\frac{a}{c}\right)^j \mu,$$

dove $\mu > 0$ è il minimo di f sulle (finite) potenze esatte di b in $[n_0, bn_0)$.

Dimostrazione (3/3)

Poiché $b^j = n/m > \frac{n}{bn_0}$ e $\frac{a}{c} = b^{\log_b(a/c)}$ con $\log_b(a/c) > 0$,

$$\left(\frac{a}{c}\right)^j = \left(b^j\right)^{\log_b(a/c)} > \left(\frac{n}{bn_0}\right)^{\log_b(a/c)}$$

Infine $\log_b(a/c) = \log_b a - \log_b c = p + \epsilon$, quindi

$$f(n) \geq \frac{\mu}{(bn_0)^{p+\epsilon}}\, n^{p+\epsilon} \qquad \text{per ogni potenza esatta } n \geq n_0,$$

cioè $f(n) = \Omega(n^{p+\epsilon})$. □

Due note sull'osservazione

- Per esempio, nell'esempio $8T(n/3) + n^2$ $(c = 8/9)$ l'osservazione dà

$$\epsilon = -\log_3 \frac{8}{9} = 2 - \log_3 8 \approx 0.107,$$

  esattamente il massimo ϵ trovato lì

- Il viceversa è falso: nell'esempio $T(n/2) + n(2 - \cos n)$, $f(n) = \Omega(n^{p+1})$ ma la condizione di regolarità non vale

Riepilogo (1/2)

Potenze esatte di b:

- albero di ricorsione: $T(n) = d\, n^{\log_b a} + g(n)$, con $g(n) = \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j)$
- identità chiave: $a^j (n/b^j)^p = n^p$, con $p = \log_b a$

Limitazioni sulla somma:

- caso 1: serie geometrica di ragione $b^\epsilon > 1$, $g(n) = O(n^p)$
- caso 2: $g(n) = \Theta(n^p S_k(L))$ con $S_k(L) = \Theta(L^{k+1})$
- caso 3: $f(n) \leq g(n) \leq f(n)/(1-c) + hMn^p$, $g(n) = \Theta(f(n))$

Riepilogo (2/2)

Ogni numero naturale n:

- con le parti intere superiori, $n/b^j \leq n_j < n/b^j + b/(b-1)$; la profondità D differisce da $\log_b n$ per una costante
- valgono le stesse limitazioni, cambiano solo le costanti; le parti intere inferiori sono analoghe
- ricorrenze comprese tra due ricorrenze in forma standard (merge sort con n qualunque)

Condizione di regolarità:

- implica $f(n) = \Omega(n^{p+\epsilon})$ con $\epsilon = -\log_b c$
- il viceversa è falso

Riepilogo: come risolvere una ricorrenza $T(n)$ (1/2)

1. È in forma standard $T(n) = a\,T(n/b) + f(n)$ con costanti $a \geq 1$, $b > 1$ e $f(n)$ asintoticamente positiva?
   - **No** $\longrightarrow$ si usi un altro metodo: iterazione, albero di ricorsione, sostituzione
   - **Sì** $\longrightarrow$ si calcoli $n^{\log_b a}$ e lo si confronti con $f(n)$

2. $f(n) = O(n^{\log_b a - \epsilon})$ per qualche $\epsilon > 0$ (polinomialmente più piccola)?
   $\longrightarrow$ caso 1: $T(n) = \Theta(n^{\log_b a})$

3. $f(n) = \Theta(n^{\log_b a} \log^k n)$ per qualche $k \geq 0$?
   $\longrightarrow$ caso 2: $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$

Riepilogo: come risolvere una ricorrenza $T(n)$ (2/2)

4. $f(n) = \Omega(n^{\log_b a+\epsilon})$ per qualche $\epsilon > 0$ (polinomialmente più grande)? Si controlli la condizione di regolarità $a\,f(n/b) \leq c\,f(n)$ con $c < 1$:
   - vale $\longrightarrow$ caso 3: $T(n) = \Theta(f(n))$
   - non vale $\longrightarrow$ il teorema non si applica

5. Nessuno dei precedenti (un **vuoto**, per esempio $f(n) = n^{\log_b a}/\log n$) $\longrightarrow$ il teorema non si applica: si usi l'albero di ricorsione

$$T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n-1} a^j f(n/b^j)$$

Base dei logaritmi: irrilevante in $\log^k n$, essenziale in $n^{\log_b a}$.